Software Architecture and Design Patterns
SoSe 2026
Prof. Dr. Jóakim von Kistowski

Exercise 0 – Collaboration, Quality and Test Recap

Learning objectives: Recap Unit testing and system testing against a network interface

Today, we will recap the practical knowledge gained during the Collaboration, Quality and Test exercises. You can find a lot of hints for this exercise in the CQT exercise sheets number 7 and 8.

As a basis for today's exercise, we will use the Fibonacci sequence.

Consider the following code that generates the *n*th (called *count* in the code) Fibonacci number:

```
public int fibonacci(int count) {
    if (count ⩽ 1) {
        return count;
    }
    return fibonacci(count - 1) + fibonacci(count - 2);
}
```

Part 1 – Implementation and Unit Tests

1. Create a Maven Project for this task. The group ID should be `de.thab.swa` and the artifact ID should be `fibonacci`.
2. Include the needed dependencies for running JUnit 5 tests in your project
3. Create a class and implement the `fibonacci` method.
4. Create a Unit Test class for the class created in 3. and write sufficient Unit tests to achieve 100% branch coverage.

Part 2 – Web Server and System Tests

Our next goal is to create a web server that serves a random Fibonacci number (from a number range of your choosing) when it is called via HTTP (as in: when it is opened in a browser).

In this exercise, use any web framework you are familiar with or *Javalin* in case none comes to mind. You can read up on how to use and include Javalin in its documentation: https://javalin.io/tutorials/maven-setup

We will the create a *system test* to test if the web server works. For this test you may use any HTTP client of your choosing. If none comes to mind, use the Jetty HTTP Client (`org.eclipse.jetty:jetty-client` on maven central, be sure to use major version 11). You can read up on the use of the Jetty HTTP Client in its documentation: https://jetty.org/docs/jetty/11/programming-guide/client/http.html

Note 1: The documentation is very thorough and includes lots of things we don't need. The most relevant parts for us are the parts on starting and stopping the HTTP client and on the use of the blocking HttpClient APIs.

Note: 2 Make sure that you use the correct HttpClient when importing it into your tests. Java provides its own HTTP Client with the same name. When importing the class, make sure that you import `org.eclipse.jetty.client.HttpClient` and **NOT** `java.net.http.HttpClient`.

1. Create a method that initializes and starts the web server with a given port. The web server should serve a random Fibonacci number when accessing the URL at "/".
2. Call this method from your main so that your server starts on port 8080.
3. Write a system test that tests your web server and its ability to serve a Fibonacci number.